תקציר מנהלים:

מטרות:

בניית תשתית אחידה לשליחת הודעות למשתמשי הקצה, ובכך לתת יכולת לכלל שירותי החברה לעמוד באותם סטנדרטים תוך כדי עמידה ביעדי אבטחה, ומוניטורינג.
ביטול כפילויות קוד, יכולת ניהול מרכזית, איפשור לכלל דומיין להתרכז בחלק העסקי ולא החלק ה"תשתיתי" של שליחת הודעות (תוך כדי עמידה וביעדי אבטחה, רגולציות, ניטור וכד')

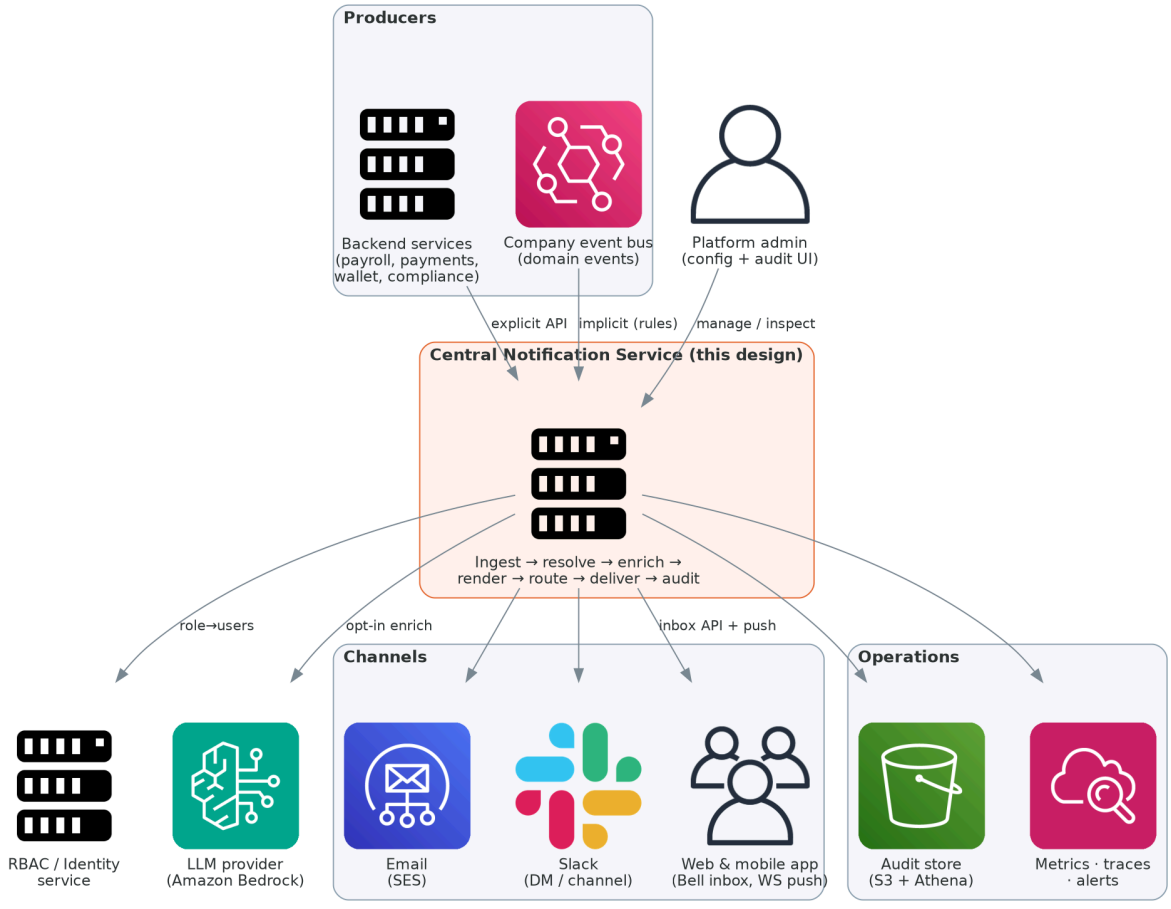
scope:

בניית API כך שכל BACKEND MS יוכל בקלות להתממשק עם השירות
תפוצת הודעות במגוון ערוצים (סלאק, אימייל, יוזר ספציפי, משתמשים בעלי תפקיד ספציפי)
תיבת הודעות מרוכזת (הודעות שלא נקראו/ רשימת הודעות/ פעולות על הודעה ועוד')
AUDIT מלא לכל המערכות

החלטות שאני בטוח לגביהן:

1. פיצול ערוץ תקשורת עם הלקוח ל"ערוץ" לוגי אחר הינו הכרחי (מניעה של בעיה בנתיב א לא "סוגרת" את נתיב ב וכי יש אפשרות לבצע סקייל "נקודתי" היה ופתאום יש "גל ענק" של ערוץ ספציפי)
2. חובה (כפול 100) שיהיה פה התייחסות לכפילויות עם idempotency כדי להבטיח שלא ישלחו 2 הודעות כפולות ללקוחות (נזק תדמיתי שווה המון והפתרון הינו קל)
3. חייב שיהיה לproducer "אחיזה" של ערך שנוכל להשתמש בו עבור סעיף 2 וחייב שזה יגיע ממנו ולא ממנגנון פנימי

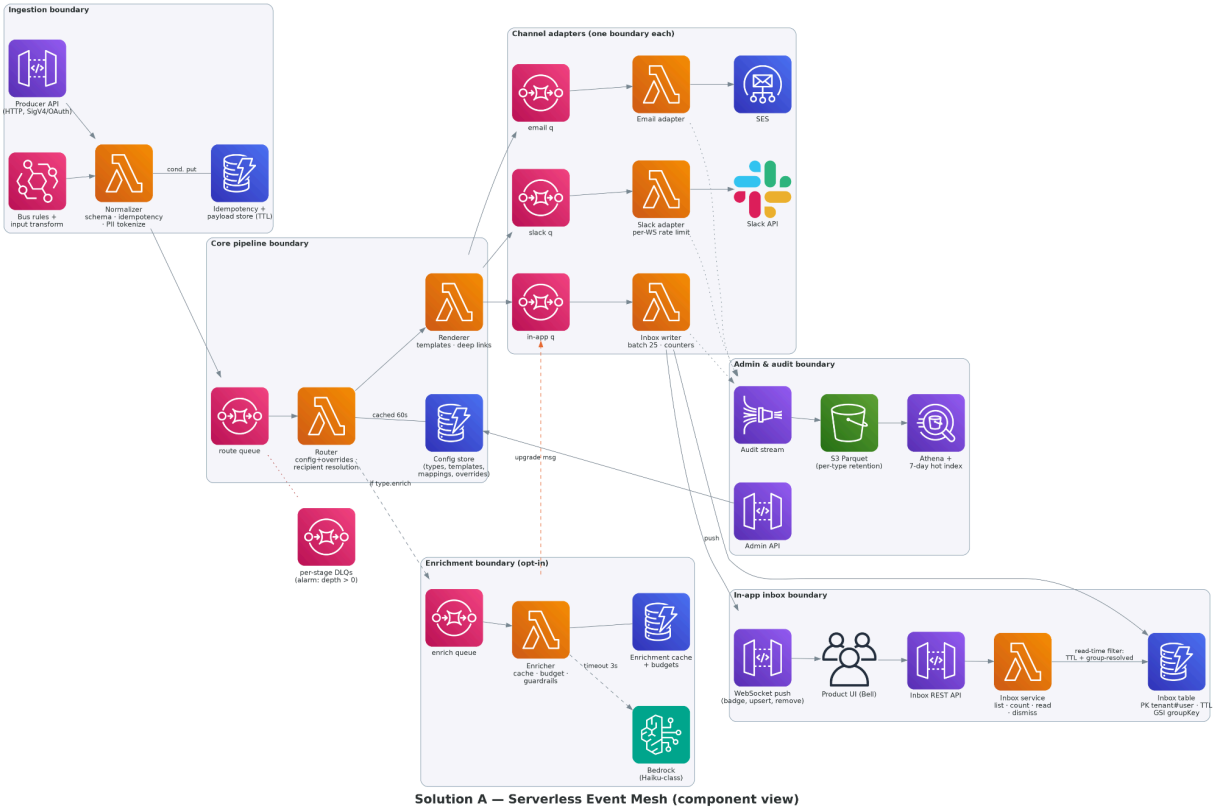
System context



System Context — Central Notification Service

Component view

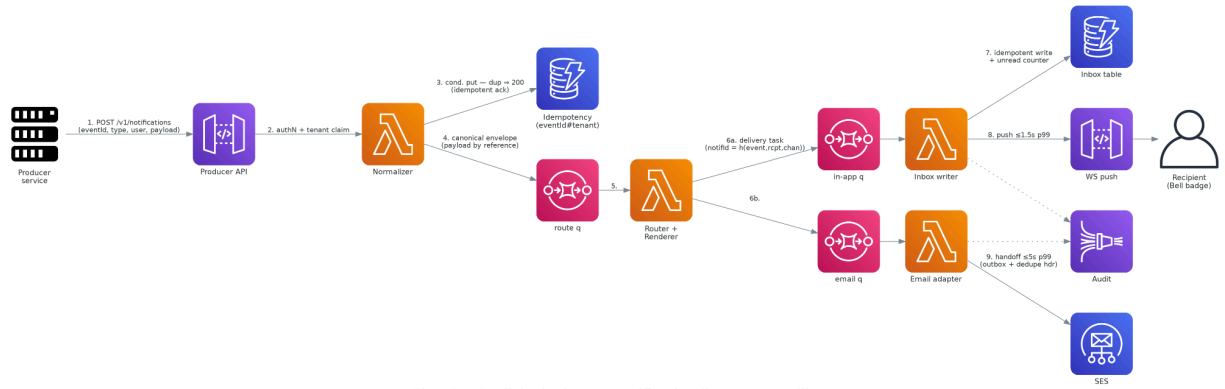
In the massive docx appears in “5. Solution A — Serverless Event Mesh”



Solution A — Serverless Event Mesh (component view)

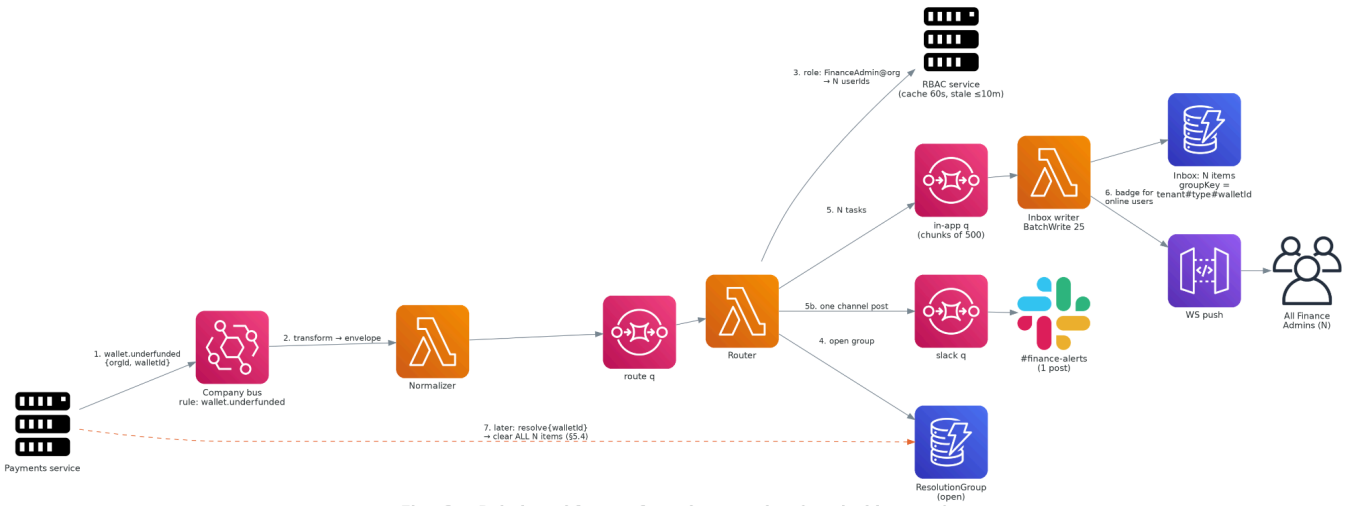
Flow diagrams

Explicit



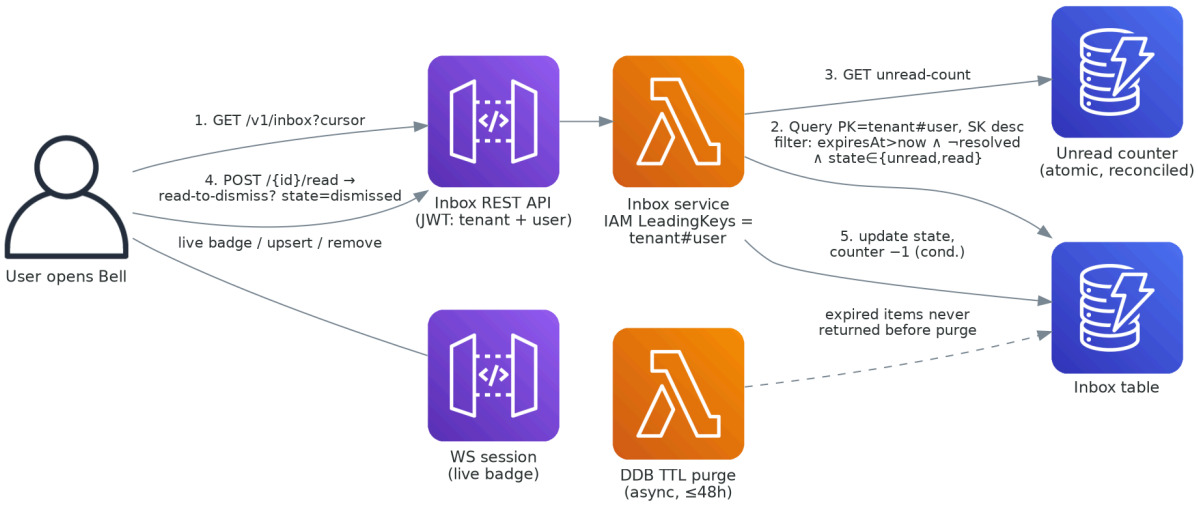
Flow 1 — Explicit single-user notification (in-app + email)

Fan-out



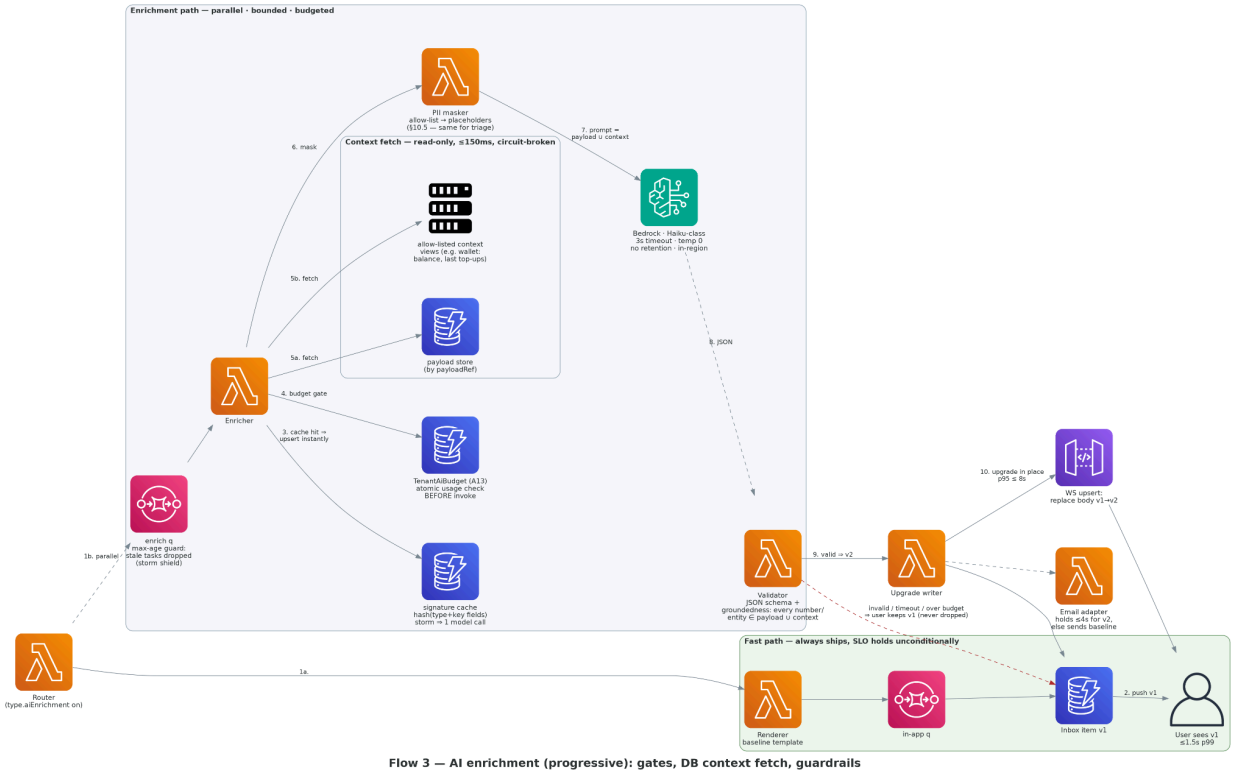
Flow 2 — Role-based fan-out from the event bus (resolvable group)

Inbox



Flow 4 — Inbox read path (list, badge, mark-read, dismiss)

AI Enrichment



Flow 3 — AI enrichment (progressive): gates, DB context fetch, guardrails

Sketch of the data model

Entity	Key fields (abridged)	Store A
NotificationType	typeId, schemaRef, channels[], ttl, resolvable, dismissOnRead, priority, aiEnrichment{}, retentionDays, deepLinkRoute	DynamoDB config
Template	typeId#channel#locale, version, body, status(draft|shadow|active)	DynamoDB
BusMapping	ruleId, action: notify | resolve, eventPattern, typeId, fieldMap, recipientSpec (notify only), origin(human | ai-triage), status(shadow|active)	DynamoDB
OrgOverride	tenant#typeId → channel toggles, digest policy	DynamoDB
Envelope (transit)	notifRequestId, eventId, tenantId, typeId, recipients[], payloadRef, entityRef?, traceparent	SQS
DeliveryTask (transit)	notifId = hash(eventId, recipient, channel), rendered body, attempt	SQS
InboxItem	PK tenant#user, SK createdAt#notifId; state, groupKey?, expiresAt(TTL), deepLink, GSI-1 groupKey	DynamoDB
ResolutionGroup	PK tenant#groupKey; state, episode, resolvedAt, sweepState; resolve-before-trigger writes a tombstone (TTL = type TTL)	DynamoDB
DeliveryRecord (outbox)	PK notifId; channel, state(sending|sent|failed|suppressed), providerMsgId, ts	DynamoDB
IdempotencyKey	PK tenant#eventId, outcomeRef, TTL 48 h	DynamoDB
AuditEvent	auditId = notifId#stage#attempt (idempotent key) · ts · tenantId · stage · outcome · channelRef · principalRef · tokensIn/Out (AI only) · reason — no payload, no free-text PII · recipient as HMAC-token	S3 Parquet (7-day hot DynamoDB index + Firehose sink)
AuditTrail retention	Same schema. S3 lifecycle = retentionDays per type. Optional Object Lock (WORM) for regulated tenants.	S3 → Athena
Connection	tenant#user → connectionIds[], TTL	DynamoDB (/ Redis optimize suggestion)
TenantAiBudget (config)	tenantId → monthlyTokenCap, perType caps, onExhaust — lives with type config (A13)	DynamoDB config
BudgetUsage	tenant#typeId#month → tokensUsed (atomic counter, checked pre-invoke)	DynamoDB

Public API surfaces (section 9 in docx)

Producer API (service-to-service)

POST /v1/notifications (idempotent via eventId; 202 {notifRequestId, duplicate:false})

```
{
  "eventId": "pay-run-8812-approval",        // producer-owned idempotency key
  "typeId": "payroll.approval.required",
  "recipients": [{"kind":"role","role":"payroll-approver","orgId":"org_912"},
                 {"kind":"user","userId":"u_army"}],
  "payload": {"payRunId":"8812","amount":182_340,"currency":"USD"},   // validated vs type schema
  "entityRef": "payrun#8812",                // groupKey seed (resolvable types)
  "options": {"channels":["inApp","email"]}   // optional narrowing only
}
```

Security — trust boundary issues to call out explicitly

orgId in the producer request body. The API sketch has the producer supply orgId. This MUST NOT be the authoritative source: a compromised producer could spoof another org. **Correct: tenantId is extracted server-side from the JWT** (or mTLS client cert for M2M). The Normalizer asserts envelope.tenantId = jwt.claims.tenantId; the body field is either absent or validated against that claim — never used as the authority.

POST /v1/notifications/resolve (202; one call clears the whole group (§5.4))

```
{ "typeId":"payroll.approval.required", "entityRef":"payrun#8812", "tenantId":"org_912" }
```

GET /v1/notifications/{notifRequestId}/status

per-recipient/channel delivery states

Event-bus mapping (config, not code)

```
{
  "ruleId": "wallet-underfunded→notify",
  "eventPattern": {"source":["payments.wallet"],"detail-type":["wallet.underfunded"],
                   "detail":{"severity":["high"]}},
  "typeId": "wallet.underfunded",
  "fieldMap": {"payload.walletId":"$.detail.walletId","payload.gapUsd":"$.detail.gapUsd",
               "entityRef":"'wallet#'+$.detail.walletId","tenantId":"$.detail.orgId"},
  "recipientSpec": [{"kind":"role","role":"finance-admin","orgId":"$.detail.orgId"},
                    {"kind":"slackChannel","ref":"#treasury-alerts"}],
  "status": "shadow"            // dry-run first; promote to active after diff review
}

// Resolve-mapping — the IMPLICIT resolver (A14 primary): domain fix event → resolve, zero producer code
{ "ruleId": "wallet-funded→resolve",
  "action": "resolve",
  "eventPattern": {"source":["payments.wallet"],"detail-type":["wallet.funds_added"]},
  "typeId": "wallet.underfunded",
  "fieldMap": {"entityRef":"'wallet#'+$.detail.walletId","tenantId":"$.detail.orgId",
               "eventTime":"$.detail.occurredAt"},    // drives the episode guard (A15)
  "status": "active" }
```

Inbox API (user-facing) and WebSocket contract

GET /v1/inbox?cursor=…&limit=25 (visibility-filtered, newest first)
GET /v1/inbox/unread-count
POST /v1/inbox/{notifId}/read (applies dismissOnRead)
POST /v1/inbox/{notifId}/dismiss
POST /v1/inbox/{notifId}/resolve (recipient-resolve (A14): clears the WHOLE group)
POST /v1/inbox/read-all
WS wss://…/inbox server→client events:

```
{ "kind":"upsert", "item":{…} } | { "kind":"remove", "notifId":"…", "reason":"resolved|expired" }
| { "kind":"badge", "unread": 7 }
```

Admin API (internal, audited)

CRUD /v1/admin/types /templates /mappings /org-overrides (versioned; cache-bust on write)

CRUD /v1/admin/tenants/{id}/ai-budget (per-tenant LLM budget — same config store as types (A13))

POST /v1/admin/templates/{id}/promote (draft → shadow → active)

Security — trust boundary issues to call out explicitly

Tenant ID in admin URL path. PUT /v1/admin/tenants/{id}/ai-budget — the {id} path param must be validated by auth middleware against the caller's JWT. A platform-admin role can target any tenant; a tenant-admin must be constrained to their own tenantId. This constraint belongs in the auth middleware contract, not in caller trust.

POST /v1/admin/dlq/{queue}/redrive (one-click, rate-limited)

GET /v1/admin/deliveries?userId=… (7-day hot audit lookup)

```
// NotificationType — doubles as the central-config illustration
{
  "typeId": "wallet.underfunded", "priority": "high",
  "channels": {"inApp":{"ttl":"P14D","resolvable":true,"dismissOnRead":false},
               "email":{"digestEligible":false}, "slack":{}},
  "aiEnrichment": {"enabled":true,"mode":"progressive","promptFields":["gapUsd","dueDate"],
                   "contextFetch":[{"view":"wallet_context","key":"$.walletId",   // §10.1: model context
                                    "fields":["balance","currency","lastTopUps"]}],
                   "timeoutMs":3000,"upgradeWindowMs":60_000,"perTypeTokenCap":2_000_000},
  "resolve": {"byRecipients": true},          // A14 — role recipients may clear the group
  "deepLinkRoute": "wallets/{walletId}/funding",
  "retentionDays": 365
}

// TenantAiBudget — lives beside the types, edited by the same admin surface (A13)
{ "tenantId":"org_912", "monthlyTokenCap": 25_000_000,
  "perType": {"wallet.underfunded": 2_000_000}, "onExhaust": "baseline+alert" }
```

Org overrides can disable a channel or route a type to digest, but cannot *widen* beyond the type's declared channels — the config model keeps platform intent authoritative and tenant intent subtractive.

AI enrichment (section 10 in docx)

Where the agent fits

Just before rendering the template (“off path” e.g. not in the hot path to avoid clogging).
The normal flow is triggered and then, in parallel, “kicks” an enrichment job in parallel.
(The “side job” upgrades the inbox item in place (conditional DB `UPDATE`, `contentVersion++`) and pushes v2 via WebSocket. (Email waits up to 4 s for enriched data, then sends whichever version is ready as the final message, no correction can be done here).
The model never sits between the producer and the delivery.

How a type opts in

In the notification type config (in the relevant “sub-config” as shown below):

```
"aiEnrichment": {
  "enabled": true,
  "mode": "progressive",
  "promptFields": ["gapUsd", "dueDate"],
  "contextFetch": [{"view":"wallet_context","key":"$.walletId","fields":["balance","lastTopUps"]}],
  "upgradeWindowMs": 60000,
  "perTypeTokenCap": 2000000
}
```

Enabled: set to **true** to enable
promptFields: is the allow-list -> only declared fields are sent to the model.
contextFetch: Enricher would require these “DB views” for enrichment context (e.g. wallet balance, last top-ups) -> fetched at enrichment time from the payload store, **not from the queue**.

Staying fast and reliable

The 4 gates below will achieve the requirement

1. **Signature cache:** hash(typeId + bucketed key fields) → cached enriched text. A storm of 10,000 identical events costs one model call; the rest are cache reads. This is the primary storm shield.
2. **Budget gate:** TenantAiBudget (same config store as the type, same admin surface) is checked atomically before every model invocation.
   (Exhaustion → baseline + admin alert. No “surprise” costs added for clients)
3. **Max-age guard:** the enrich queue has a message TTL (**upgradeWindowMs**, default 60 seconds, defined per notification type). A backed-up queue *drains by discarding* stale upgrade work, not by calling the model faster.
   (Audited as dropped:max-age)

4. **3-second hard timeout + circuit breaker:** timeout (or repeated failures) → “baseline” (template) survives; breaker opens and the enrichment step is skipped entirely until a probe succeeds. Drop is not on the ladder.

Cost and safety

Storm cost: the signature cache collapses any storm to a unique-shape count (~20–200/day empirically). Per-type and per-tenant token caps enforce a hard ceiling before invocation of the model call.

PII in prompts: prompts are assembled from the declared `promptFields` allow-list only, then run through a PII masker that replaces names/emails/identifiers with stable placeholders (`{{NAME}}`, `{{AMOUNT}}, etc.`).
The AI model sees masked text, real values are substituted into the result after validation. Prompts and outputs are never persisted (I thought about adding an evaluation/ monitoring but it’s out of scope for now).
* Only a salted content hash and token counts in the audit trail.
* Bedrock is configured with no data retention, in-region.

Hallucination: two checks.

1. Temperature 0 and a strict JSON output schema (structured generation guardrail).
2. G**roundedness echo-check**: every number, date, and entity in the model's output must exist in `payload ∪ fetched context` —> any mismatch discards the enrichment completely, *not the notification*. The model can only say things the input already contained.

Enriched content at rest: the generated text lives inside the user’s inbox (*InboxItem*), so it automatically inherits the tenant CMK encryption, the item has an expiration (*expiresAt*) TTL, and all visibility rules (nothing AI-written outlives or out-travels the notification it belongs to).

One way door ADRs (section 14 in docs)

Choreographed queues over central orchestration

Context: Every notification must move through ingestion → routing → rendering → per-channel delivery.
Each hop can fail independently, must retry safely, and must not couple failure in one channel to another.
The two obvious topologies are:

1. Central orchestrator (Step Functions) that calls each stage in sequence and owns the state machine
2. Choreography, where each stage reads from its own queue, does its work, and writes to the next.

Decision*:* Choreography via SQS + Lambda. Each stage owns a queue and a DLQ. No central state machine.

Alternatives considered:

- **AWS Step Functions (Express Workflows)**
    - **Goodies:**
    - clean visual flow
    - built-in retry/timeout
    - state visible in one place.
    - **Rejected for two reasons**
    - 1-> cost at volume (115M state transitions/day ≈ $2,900/day at list price), and coupling.
    - 2-> Blockage:
      Standard workflow that drives all channels means a Slack adapter failure can ***block*** the inbox writer. Making channels independent requires parallel branches and error handling that recreates queue semantics inside the orchestrator, at 100× the cost.
- **Single shared queue:** simpler to operate, one DLQ. Rejected because a poison message or a slow channel (Slack rate-limit backpressure) creates head-of-line blocking for all tenants and all channels simultaneously.

Consequences: End-to-end "what happened to notification X?" requires joining audit records across stages rather than reading one execution's history -> accepted, mitigated by the audit trail and the delivery status API. ***Choreography*** is a one-way door: the data model, retry policy, and observability tooling are all built around per-stage queues; folding an orchestrator back in later would be a full re-architecture. The cost saving (~$2,900/day) justifies accepting that constraint.

Pooled tenancy with IAM-enforced key isolation; enterprise tier as silo cells

Context: The service must enforce hard tenant isolation (no cross-tenant data leak is acceptable).
The naive options are:

1. One database/table per tenant (pure ***silo***)
2. Shared tables with application-level tenant filtering.
3. Shared infrastructure with cryptographic isolation at the storage level, plus a separate physical cell for tenants who require it contractually.

Decision: *Pooled DynamoDB* tables for standard tenants; every key is prefixed `tenantId#...`while inbox roles carry an IAM `dynamodb:LeadingKeys` condition bound to the JWT's `tenantId` claim. ***Enterprise*** tenants are stamped as isolated cells (separate table set, separate KMS key, separate IAM role boundary) using the same IaC template (the *same design*, instanced).

Alternatives considered:

- **Application-layer (`WHERE tenantId = ?)` -> shared tables, no storage enforcement:** one missing WHERE clause is a data leak. Indefensible for payroll content under audit. ***Rejected***.
- **Silo for everyone:** eliminates cross-tenant risk. ***Rejected*** because it multiplies infrastructure proportionally with tenant count, creates a per-tenant operational burden (migrations, quota management, monitoring), and makes the enterprise cell story identical in cost to the standard story - removing the commercial differentiation between tiers.
- **Row-level encryption per tenant, pooled table:** protects data at rest but does not prevent a buggy query from *returning* another tenant's ciphertext (even if unreadable). LeadingKeys stops the query from executing at all.

Consequences: Pooled standard tenants share throughput capacity and must be defended against noisy neighbors at the Lambda concurrency and queue level (per-tenant ingest rate limits, per-tenant enrichment budgets). ***Enterprise*** cell stamping means the cell becomes a unit of operations: upgrades, migrations, and quota raises must be applied per cell — bounded by the IaC automation, but non-zero. This is a one-way door because the inbox data model (`PK = tenantId#userId`) cannot be migrated to a different isolation scheme without a full data migration; choosing it at the start is cheaper by an order of magnitude than changing it later.

Failure modes and operability (section 13 in docx)

What breaks first is almost always external: SES bounce rate spikes (domain reputation), Slack 429s from a large fan-out, or Bedrock latency under load. Internal failures are rarer and contained by design.

What breaks	How you know	What the on-caller sees
SES sending rate hits quota	CloudWatch `Sending quota utilization > 80%` alarm	Email DLQ depth rising; in-app unaffected. Fix: emergency quota raise or throttle email channel
Slack 429 storm (large fan-out)	Per-workspace queue age alarm	Slack messages delayed, in-app fine. Fix: token bucket already retrying; check workspace ceiling
RBAC service down	Circuit breaker open alarm; route-q oldest-message-age rising	Fan-out notifications stalled (parked, not dropped). Fix: stale membership serves up to 10 min; beyond that the queue drains when RBAC recovers — idempotent
DLQ depth > 0 (any stage)	DLQ alarm at depth = 1 (immediate)	One notification stuck. Fix: inspect message, fix root cause, one-click idempotent redrive
Lambda cold-start storm (burst onset)	p99 latency alarm on inbox write	In-app latency rises toward ~0.95 s (still under SLO). Self-resolves in ~60 s as environments warm
DynamoDB on-demand ramp lag	Write throttle metric	Brief slow inbox writes at burst onset. Provisioned base absorbs sustained; on-demand covers delta
Bedrock slow / over budget	Enrichment fallback % alarm; budget burn metric	Users get v1 baseline — no user-visible failure. Fix: adjust budget or breaker threshold

Poison message	DLQ alarm at depth = 1	One notification stuck. Fix: inspect, fix, redrive
Badge counter drift	Weekly reconciliation drift-rate metric spikes	Wrong badge number for dormant user. L1 self-heals on next inbox open; drift metric pages on pattern
Audit trail gap	Daily reconciliation alarm > 0.1% drift	Compliance gap. Fix: Firehose retry + re-emit from DeliveryRecord source of truth

Golden dashboard has five numbers: ingest rate vs baseline · all DLQ depths (target: 0) · in-app p99 · enrichment fallback % · SES quota burn %. Everything else is a runbook link away.

Risks and open questions (section 15 in docx)

My mistake — I dropped R4 when you said "there are 10 not 11" in the previous message, but you were correcting me from my earlier chat reply that only listed 8. R4 belongs in the table. Here it is complete with all 11:

Rank	Risk	Severity	Likelihood	External dependency	Score	Status
1	R6 — Bus schema drift silently breaks mappings and implicit resolver	High	High	High	27	Live — needs schema registry contract with bus team
2	R2 — RBAC staleness window is a potential info-disclosure for sensitive types	High	Med	High	18	Needs security sign-off before go-live
3	R11 — AI triage: wrong/unwanted rules, promotion ownership unresolved	Med	Med	High	12	Open — needs product + platform decision
4	R1 — Slack rate limits on large fan-outs	Med	High	Med	12	Mitigated — remaining lever is product (summary threads)
5	R9 — eventTime producer contract and resolveQuietPeriod defaults not chosen	Med	Med	Med	8	Residual from R9 close — producer onboarding checklist
6	R7 — Per-user preferences vs org overrides collision (phase 2)	Low	High	Med	6	Pre-wired — schema seam reserved, no action needed now

7	R3 — Fan-out above 5,000 recipients	Med	Med	Low	4	Decided: paced waves + over-cap alarm
8	R5 — LLM cost default caps per pricing tier not yet chosen	Low	Med	Med	4	Residual — product/finance input needed
9	R8 — WebSocket scale past ~100k concurrent connections	Med	Low	Low	2	Planned — quota raises + jittered reconnect + tenant-hash sharding
10	R4 — Email delivery observability	Low	Low	Low	1	Closed — SES event publishing, alarms on bounce/complaint/DLQ
11	R10 — Template versioning across in-flight retries	Low	Low	Low	1	Decided — tasks pin templateVersion at render time